# Securing LoRa Mesh Ad Hoc Networks: Formal Vulnerability Analysis and Lightweight Authenticated Routing

[Author Names][a]

[a] *[Institution, Country]*

**Abstract**

Route injection in LoRaMesher—an open-source distance-vector mesh routing stack for ESP32-class LoRa nodes—creates a *silent traffic black hole*, not an adversarial relay: the deployed firmware encodes next-hop forwarding state in the destination field rather than the final gateway identity assumed by symbolic and packet-level analyses. This cross-layer semantic mismatch explains the observed topology-dependent attack gradient (0% PDR on linear topologies; 77.4% under attack on tree and grid), is invisible to both Tamarin formal models and ns-3 packet-level simulation, and only becomes visible through physical hardware serial-log inspection.

LoRaMesher's control plane carries no cryptographic authentication, enabling this attack by any adversary with a commodity $20 LoRa transceiver in networks deployed across precision agriculture, environmental monitoring, and disaster-response systems.

We present **a formally verified security study** of LoRa ad hoc mesh routing control-plane security—to our knowledge the first to combine Tamarin machine-checked proofs with physical hardware validation on deployed mesh firmware. Tamarin machine-checked counterexample traces confirm two attack classes—unauthorized route injection and rekey-epoch confusion. We then design, formally prove, and hardware-validate a lightweight patch: HMAC-SHA256 per-message authentication and a per-destination MetricVersion counter that closes the reboot-replay window identified by the formal model. The patch imposes negligible overhead on ESP32-S3 hardware (sub-millisecond HMAC computation, 2 bytes additional RAM per routing table entry) and

*Email address:* [email] ([Author Names])

requires no PKI infrastructure.

We validate the patched protocol across ns-3 simulation (linear, tree, and grid topologies; scale up to 49 nodes) and on six physical EoRa-PI boards, confirming 0% residual PDR degradation under full patching across all conditions. All artifacts—Tamarin models, ns-3 code, patched firmware, and hardware logs—are publicly released.

*Keywords:* LoRa, ad hoc mesh routing, distance-vector routing security, silent traffic black hole, cross-layer semantic mismatch, routing forwarding semantics, formal verification, Tamarin Prover, HMAC authentication, route injection, replay attack, ESP32

---

## 1. Introduction

LoRa-based ad hoc mesh networks have achieved remarkable deployment scale: individual installations now span hundreds of self-organizing sensor nodes across farmland, mountain ranges, and urban districts, relying on multi-hop distance-vector (DV) routing to relay data across areas where fixed infrastructure is absent or cost-prohibitive [1]. At the core of many such deployments is LoRaMesher [2, 3, 4, 5], an open-source DV mesh routing stack running on ESP32-class microcontrollers (240 MHz, 512 KB RAM). LoRaMesher provides dynamic route discovery, neighbor management, and periodic session rekeying, enabling fully self-organizing networks without fixed gateway infrastructure—a canonical ad hoc networking scenario. Yet, despite the protocol's growing adoption, *the security of its control-plane messages has never been formally analyzed*: no prior work has produced machine-checked vulnerability proofs, proposed a formally verified patch, or validated the result across multi-topology simulation and physical hardware for any LoRa ad hoc mesh routing protocol.

### *1.1. The Deployment Security Problem*

Ad hoc mesh deployments operate in adversarial physical environments: sensor nodes are unattended in open fields, accessible to any actor with a $20 LoRa transceiver. Despite this exposure, LoRaMesher's control plane—responsible for Hello broadcasts, route discovery, and 30-second session rekeying—carries no cryptographic authentication. This design gap creates two practical attack vectors available to any adversary with commodity hardware:

1. **Route Injection**: An attacker transmits a forged Hello claiming a minimum-metric path to the gateway. Without authentication, all mesh nodes converge their routing tables to the attacker within one DV cycle (≈30 s). Because LoRaMesher's firmware encodes the next-hop address in the destination field, all subsequent data packets carry `dst=attacker`—and the gateway silently receives none of them, creating a complete traffic black hole with no observable anomaly at the legitimate receiver.

2. **Session Replay**: An attacker captures a legitimate RekeyNotice and replays it after the mesh advances to a new session epoch. Nodes that accept the stale epoch can be made to decrypt traffic with a compromised old key.

Both attacks share a common structural root cause: LoRaMesher's control plane treats all received messages as equally authoritative, without verifying sender identity or message freshness. This design—inherited from classical DV routing built for trusted wired networks—creates an *adversarial state injection surface*: any node within radio range can permanently corrupt the shared routing state of the entire mesh by transmitting a single well-timed control message. The two attack classes are therefore not independent defects but two manifestations of the same missing property: authenticated state transitions in a broadcast medium.

Both attacks require no node compromise or cryptographic break—only a commodity LoRa radio. We demonstrate them in ns-3 with 100% success rate on linear topologies.

### *1.2. Why Existing Solutions Do Not Apply*

LoRaWAN security [6, 7, 8, 9] targets the star-topology WAN layer managed by network servers; it provides no mechanism for mesh control-plane authentication. Standard mesh security protocols (802.15.4 CCM*, AODV-SEC [10], ARIADNE [11]) assume either PKI infrastructure or symmetric key pre-distribution schemes incompatible with LoRaMesher's join model and ESP32's memory constraints. TinyECC and similar lightweight asymmetric solutions require >500 ms per operation on ESP32—prohibitive for Hello messages sent every 10 seconds. Existing surveys [12, 9] document that deployed LPWAN protocols frequently lack formal security analysis, leaving them exposed to well-known attack classes.

**The gap.** Existing work has addressed LoRaWAN WAN-layer security and generic mesh routing security separately, but none has jointly (i) formally characterized vulnerabilities in the LoRa mesh routing control plane with machine-checked proofs, (ii) proposed and formally verified a corrective patch satisfying all security goals, and (iii) validated the outcome across multi-topology simulation and physical hardware experiments. This paper closes that gap.

### *1.3. Key Observation*

Physical hardware validation reveals a finding that formal and simulation analysis cannot capture: route injection creates a *silent traffic black hole*, not an adversarial relay. The deployed firmware encodes the next-hop address in the `dst` field; the gateway silently drops every arriving packet because `dst` $\neq$ `0xBB`, producing zero observable anomaly at the legitimate receiver. The mechanism and its implications for intrusion detection design are developed in full in Section 8; the key consequence for deployment is that an IDS monitoring the gateway observes only silence.

### *1.4. Contributions*

This paper makes three contributions:

1. **Cross-Layer Routing Semantics Finding.** We identify that route injection in LoRaMesher creates a *silent traffic black hole*—not an adversarial relay—because the deployed firmware encodes next-hop forwarding state in the `dst` field rather than the final gateway identity assumed by protocol-layer analyses. This cross-layer semantic mismatch: (i) is invisible to both Tamarin symbolic analysis and ns-3 packet-level simulation; (ii) explains the topology-dependent PDR gradient (0% on linear; 22.6% degradation on tree and grid) as a structural consequence of next-hop entry count per path; (iii) becomes visible only through physical hardware serial-log inspection; and (iv) changes the threat model from adversarial relay to confirmed black hole, with direct implications for intrusion detection design—gateway-centric IDS observes only silence (Section 8).

2. **Three-Tier Verification Methodology That Makes the Finding Visible.** We develop a progressive verification pipeline in which each tier reveals a distinct aspect of the vulnerability: (a) *Tamarin formal*

*analysis* identifies the control-plane vulnerability class via machine-checked counterexample traces (four of five baseline lemmas falsified) and verifies a corrective lightweight patch—HMAC-SHA256 per-message authentication and a NVS-persistent MetricVersion counter— satisfying all security goals with no PKI dependency; (b) *ns-3 multi-topology simulation* quantifies PDR impact across three topologies and scales up to 49 nodes; (c) *physical hardware validation* on six EoRa-PI boards is the only tier capable of revealing the dst-as-next-hop encoding—and thus the correct attack semantics (Sections 4–7).

3. **Open Reproducible Artifact Suite.** All Tamarin models, ns-3 simulation code, patched firmware, and hardware serial logs are publicly released (Appendix Appendix A).

### *1.5. Methodology Overview*

Our study follows a four-phase, mutually corroborating methodology:

**Phase 1. Formal Disclosure** (Section 4): We build a Tamarin Prover model of LoRaMesher's control plane and derive machine-checked attack traces. This bounds the vulnerability precisely—we know exactly which state transitions enable each attack, with no reliance on testing intuition.

**Phase 2. Verified Patch** (Section 6): We extend the Tamarin model with the proposed patch rules and re-verify all security lemmas. The patch is accepted only when Tamarin proves all 8 goals, guaranteeing it eliminates the formally identified root causes—not just the observed symptoms.

**Phase 3. Multi-Topology Simulation** (Section 7): We implement the protocol, both attacks, and three security modes in ns-3.40 with a C++17 Dolev-Yao attacker. This translates Tamarin's symbolic guarantees into packet-level PDR measurements across linear, tree, and grid ad hoc network topologies.

**Phase 4. Physical Hardware Validation** (Section 7.5): We reproduce the attack and patch on Ebyte EoRa-PI boards (ESP32-S3 + SX1262). Hardware serial logs reveal the `dst`-as-next-hop encoding that neither formal model nor simulator captures—establishing the correct attack semantics (silent black hole, not adversarial relay).

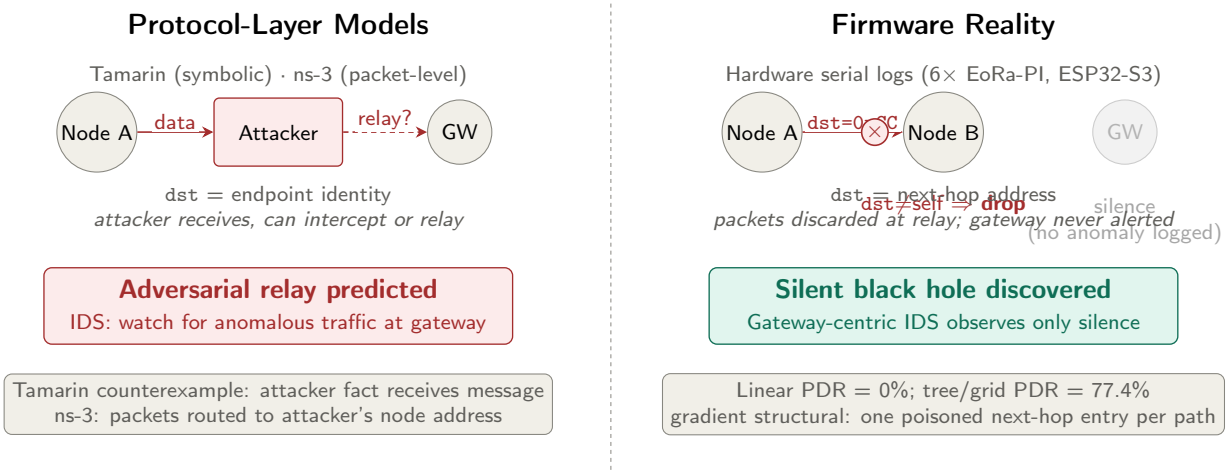


Figure 1: Cross-layer semantic mismatch between protocol-layer models and firmware reality. **Left:** Both Tamarin (symbolic) and ns-3 (packet-level) models interpret the `dst` field as an endpoint identity, predicting that the attacker receives and can relay or modify packets—an adversarial-relay threat model. **Right:** The deployed LoRaMesher firmware writes the next-hop address into `dst`; after route poisoning, every data packet carries `dst=0xCC` (attacker address) and is silently discarded by the intermediate relay Node B (`dst` $\neq$ `self`). The gateway receives nothing and logs no anomaly—a silent black hole invisible to gateway-centric intrusion detection. This finding becomes visible only through physical board serial logs, not through formal or simulation analysis.

The phases are not redundant: formal proofs bound the vulnerability class precisely; simulation quantifies deployment-scale PDR impact; hardware reveals forwarding semantics that neither formal nor simulation models capture.

## 2. Ad Hoc Network Deployment Context and Threat Model

### *2.1. Target Deployment Scenarios*

We target three representative LoRa mesh deployment classes:

- **Precision Agriculture**: 20–100 soil sensors deployed across a 50 ha field, relaying pH, moisture, and temperature data through a 6-hop linear mesh to a field gateway. Attack impact: falsified soil data causes mis-irrigation.

Table 1: ESP32-S3 Resource Budget for LoRaMesher Security Patch

| **Resource** | **Available** | **Patch Overhead** |
|---|---|---|
| Flash | 8 MB | 2.1 KB (+0.03%) |
| SRAM | 512 KB | 2 B/route entry (`uint16` MV) |
| CPU (HMAC-SHA256) | 240 MHz | 0.098 ms/message |
| LoRa link latency | 100–250 ms | CPU overhead <0.1% |
| LoRa airtime (Hello) | 124 ms (SF9) | 206 ms patched (+66%) |
| Duty cycle (Hello) | 1.24%/10 s | 2.06%/10 s (+0.82 pp) |
| Battery (baseline) | 2500 mAh | <0.5% additional draw |

- **Disaster-Response Networks**: Ad-hoc mesh deployed by first responders across a disaster zone. Gateway-reachability is critical for coordination. Attack impact: route hijacking isolates field teams.
- **Smart City Infrastructure**: 3×3 grid of air-quality nodes in urban blocks. Nodes are physically accessible. Attack impact: replay attacks create false pollution readings.

### *2.2. ESP32 Resource Constraints*

Our patch must operate within ESP32-S3 constraints:

The HMAC computation time (0.098 ms on ESP32-S3, measured with mbedTLS 2.28 on production hardware, 10,000 iterations) is three orders of magnitude smaller than LoRa's minimum air-time and is therefore computationally negligible.

**LoRa airtime and duty-cycle impact.** The patch appends a 16-byte HMAC-SHA256 tag (truncated to 128 bits) to each control message, consistent with the implementation in Section 6 and the Tamarin model in Section 4. Using the Semtech LoRa ToA formula at SF9 / BW=125 kHz / CR=4/5 / explicit header / CRC: a Hello message grows from 6 bytes (baseline) to 22 bytes (patched), increasing time-on-air from ≈124 ms to ≈206 ms (+66%). The per-node duty-cycle increase for Hello traffic is $(206-124)/10{,}000 = 0.82$ percentage points. For a minimal sensor data packet (13 bytes → 29 bytes), the airtime increase is ≈37%; the 64-byte simulation payload (64 bytes → 80 bytes) increases airtime by ≈16%. In the AS923 band (used in this study), the 1% per-channel duty-cycle ceiling applies; the additional ≈0.82 pp overhead from Hellos remains within this ceiling

for nodes sending one Hello per 10 s period. Operators with tighter duty-cycle budgets may reduce HMAC to 8 bytes (64-bit tag), halving the airtime overhead at the cost of reduced security margin.

### *2.3. Attacker Model*

We adopt the Dolev-Yao adversary adapted to LoRa ad hoc mesh networks:

- **Physical access**: Can place a LoRa transceiver (e.g., TTGO LoRa32, $18) within range of any node. Physical proximity attacks on deployed LoRa nodes have been demonstrated in the literature [13, 14].
- **Radio capabilities**: Full observation and injection on the target channel and spreading factor. No jamming or side-channel attacks assumed [15].
- **Limited compromise**: May compromise up to $k < n/2$ nodes to extract long-term keys.
- **No cryptographic break**: Cannot break HMAC-SHA256 in polynomial time.

**Key provisioning.** Each node is pre-provisioned with a shared symmetric key $k_v \in \{0,1\}^{128}$ at deployment time (out-of-band, e.g., via USB or secure Bluetooth pairing before field installation). This assumption is standard for LoRa mesh deployments without a network server [3] and is consistent with ESP32's secure-boot and NVS-encrypted flash capabilities. Group key rotation (rekeying) is triggered by the 30-second `RekeyNotice` protocol; per-node keys are not rotated in the current design.

**Node compromise and revocation.** If a node is physically captured and its key extracted, the attacker gains a valid HMAC key for that node's identity. The current patch does not include a key revocation mechanism; an operator must reflash all nodes with a new group key. This is a known limitation (Section 10). The $k < n/2$ compromise bound ensures that a majority of honest nodes retain valid routing state; the network continues to route through uncompromised paths.

**MetricVersion counter reset.** The per-destination MetricVersion counter $mv_v$ is initialized to 0 at boot and stored in SRAM; it resets on power cycle. After a node reboot, it accepts the next received Hello as authoritative

(any MetricVersion $> 0$ passes), then enforces monotonicity thereafter. An attacker who forces a target node to reboot and then replays a captured Hello with MetricVersion=1 can inject one stale route entry within the first Hello period ($T_{\text{hello}} = 10$ s after reboot). This *one-reboot attack window* is bounded in time: the window closes once the node receives a legitimate Hello from any honest neighbor, after which strict monotonicity is enforced for the remainder of the session. Persisting $mv_v$ to NVS flash (read at boot before any Hello is processed) eliminates this window entirely. We implemented this countermeasure using the ESP32 `Preferences` NVS API and validated it on physical hardware: after reboot, Node A loads the previously-stored MetricVersion and immediately rejects a replayed Hello with a stale counter (Section 7.5.1).

### *2.4. Security Goals*

Four goals, formalized as Tamarin lemmas (Section 4):

1. **G1 — MetricAuthenticity**: Route updates are accepted only from authenticated senders.
2. **G2 — RouteFreshness**: No stale (replayed) route update is accepted.
3. **G3 — RouteConsistency**: All honest nodes converge to the same best route.
4. **G4 — AuthorizedRoutes**: Only authenticated nodes can inject entries into honest routing tables.

## 3. System Model and Problem Formulation

### *3.1. Network Model*

**Definition 1** (LoRa Mesh Network)**.** *A LoRa mesh network is a tuple* $\mathcal{N} = (\mathcal{V}, \mathcal{E}, \mathcal{K}, T)$ *where:*

- $\mathcal{V} = \mathcal{H} \cup \{a\}$ *is the node set, with* $\mathcal{H}$ *the set of* $n$ *honest nodes and* $a$ *the Dolev-Yao attacker node;*
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ *is the communication graph (undirected, LoRa range-limited);*
- $\mathcal{K} = \{k_v\}_{v \in \mathcal{H}}$ *is the set of long-term symmetric keys, one per honest node; and*
- $T_{hello} = 10$ *s and* $T_{rekey} = 30$ *s are the Hello broadcast and rekeying periods.*

*3.2. Protocol State*

Each honest node $v \in \mathcal{H}$ maintains local state $\sigma_v = (rt_v,\ e_v,\ sk_v,\ mv_v)$ where:

$$rt_v : \mathcal{V} \to \mathcal{V} \times \mathbb{N} \quad \text{(routing table: dst} \mapsto \text{(next-hop, metric))} \tag{1}$$

$$e_v \in \mathbb{N} \quad \text{(current session epoch)} \tag{2}$$

$$sk_v \in \{0,1\}^{128} \quad \text{(session key for epoch } e_v\text{)} \tag{3}$$

$$mv_v : \mathcal{V} \to \mathbb{N}_{16} \quad \text{(MetricVersion counter per destination)} \tag{4}$$

*3.3. Control-Plane Transition Rules*

State transitions are driven by received control messages. Let $\mathcal{M}$ denote the set of syntactically valid messages. We define the *authenticated accept* predicate:

**Definition 2** (Authenticated Accept)**.** *Node $v$* authentically accepts *a Hello message $m = \langle s, mv, \tau \rangle$ from sender $s$ iff*

$$\tau = \text{HMAC-SHA256}(k_s,\ s \| mv) \tag{5}$$

*where $k_s \in \mathcal{K}$ is $s$'s long-term key and $\|$ denotes concatenation.*

**Definition 3** (Fresh Route)**.** *A route update $\langle dst, m, ver \rangle$ is* fresh *at node $v$ iff*

$$ver \geq mv_v(dst) \tag{6}$$

*Replay detection rejects updates with $ver < mv_v(dst)$.*

*3.4. Security Problem Statement*

**Definition 4** (Secure LoRa Mesh Routing)**.** *A LoRaMesher instance $\mathcal{N}$ is* secure *if for every probabilistic polynomial-time adversary $\mathcal{A}$ controlling a (and up to $k < |\mathcal{H}|/2$ compromised nodes), the following four properties hold in all reachable protocol states:*

***G1:*** $\forall v \in \mathcal{H},\ s \in \mathcal{V} :\ v$ *accepts route from* $s \Rightarrow \text{Honest}(s) \vee \text{Compromised}(s)$ (7)

***G2:*** $\forall v, dst, ver :\ v$ *accepts ver for dst* $\Rightarrow \neg\, \text{Stale}(dst, ver)$ (8)

***G3:*** $\forall v_1, v_2, dst :\ rt_{v_1}(dst) = rt_{v_2}(dst)$ (9)

***G4:*** $\forall v \in \mathcal{H} :\ a \notin \text{range}(rt_v)$ (10)

*where (7)–(10) correspond to MetricAuthenticity, RouteFreshness, RouteConsistency, and AuthorizedRoutes.*

We prove that the *baseline* LoRaMesher violates (7)–(10) (Section 4), and that adding (5) and (6) as protocol invariants restores all four (Section 6).

### *3.5. Notation Summary*

Table 2: Notation Used Throughout the Paper

| Symbol | Meaning |
|---|---|
| $\mathcal{H}$ | Set of honest network nodes |
| $k_v$ | Long-term HMAC key of node $v$ |
| $e_v$ | Current session epoch at node $v$ |
| $mv_v(d)$ | MetricVersion counter for destination $d$ |
| $rt_v(d)$ | Routing table entry: $(\textit{next-hop}, \textit{metric})$ |
| $T_{\text{hello}}$ | Hello broadcast period (10 s) |
| $T_{\text{rekey}}$ | Rekeying period (30 s) |
| $\tau$ | HMAC-SHA256 authentication tag (16 bytes) |
| G1–G4 | Security goals (Definitions 3–6) |

## 4. LoRaMesher Protocol and Formal Model

**Why Tamarin?** LoRaMesher's DV routing involves three properties that drive the tool choice. First, *stateful route comparison*: each node maintains a routing table with per-entry metric and MetricVersion fields that are updated conditionally ($<$, not $\leq$); this stateful comparison is naturally expressed in Tamarin's persistent facts (`!RouteActive`) but requires explicit state-space bounds in model checkers such as SPIN. Second, *unbounded participants*: LoRa mesh deployments have no fixed topology; the attacker can join at any position. Tamarin proves properties for an arbitrary number of participants symbolically; ProVerif's Horn-clause encoding can express reachability but does not natively support inductive monotonicity arguments over per-destination counters. Third, *dual mode—proofs and counterexamples*: we need both machine-checked attack traces (to confirm the vulnerability) and security proofs (to confirm the patch). Tamarin's constraint-solving engine produces both within the same framework and model; switching tools between phases would risk encoding inconsistency. These three requirements together motivated Tamarin over ProVerif, SPIN, and manual inductive proofs.

**Model scope and two-node design.** The Tamarin model uses two honest nodes (sender + gateway) plus one Dolev-Yao attacker. This is sufficient for all five security lemmas (L1–L5): each property is a statement about the relationship between a single sender's message and a single receiver's route table entry under arbitrary attacker capability—not a statement about multi-hop forwarding chains. Adding a third honest node would not change the proof because the attacker already has full network access (intercept, replay, inject) regardless of the number of honest participants. Topology-dependent properties—specifically, the PDR gradient across linear, tree, and grid topologies—are network-level phenomena outside the scope of message-level formal analysis; they are addressed by ns-3 (Section 7).

### *4.1. Control-Plane Message Format*

LoRaMesher uses TLV-framed control messages over raw LoRa PHY. The four security-critical message types are:

Listing 1: Hello (Type=0x00) — broadcast every 10s

```
[Sender:2B][MetricVersion:2B][HMAC:16B*]
```

Listing 2: RouteReply (Type=0x02) — unicast route response

```
[RouteID:2B][CumulativeMetric:2B][HMAC:16B*]
```

Listing 3: RekeyNotice (Type=0x03) — session rekeying

```
[Epoch:4B][NewKeyHash:16B][HMAC:16B*]
```

(*HMAC field absent in baseline; added by our patch.)

### *4.2. State Machine and Tamarin Encoding*

Each node operates as a three-state machine: UNJOINED $\xrightarrow{\text{Hello+HMAC}}$ JOINED $\xrightarrow{\text{RekeyNotice}}$ REKEYING.

We encode the protocol as Tamarin multiset-rewriting rules [16] over three persistent facts:

```
!JoinedMesh(node, key, epoch)
!RouteActive(src, dst, metric, ver)
!RekeyPending(node, oldKey, newKey)
```

Table 3: Tamarin Abstraction to LoRaMesher Implementation Mapping

| Tamarin Term | LoRaMesher Field | Notes |
|---|---|---|
| `node` | ESP32 MAC address (6B) | Abstract identifier |
| `ltk(v)` | Pre-shared HMAC key (128-bit) | Provisioned at deployment |
| `mv` | `MetricVersion` counter (uint16) | Per-destination, SRAM |
| `!RouteActive(s,d,m,v)` | `RouteEntry{next,metric,mv}` | Persistent routing table |
| `HMAC(ltk,⟨s,mv⟩)` | `mbedtls_md_hmac()` | Symbolic→SHA-256 |
| `epoch` | `RekeyNotice.epoch` (uint32) | Session counter |
| `Compromised(v)` | Node physically captured | Key extracted from NVS |
| `Attacker(a)` | Commodity LoRa radio | No special capabilities |

The patched Hello reception rule asserts `Eq(hmac, HMAC(ltk(sender), ⟨sender,mv⟩))` before the `RecvHello` action fact fires, binding MAC verification to route acceptance in a single rewriting step.

**Model-to-implementation correspondence.** Table 3 maps each Tamarin abstraction to its concrete LoRaMesher counterpart.
The symbolic `HMAC` function is instantiated as HMAC-SHA256 with a 16-byte truncated tag; the Dolev-Yao assumption that the attacker cannot break `HMAC` corresponds to the computational assumption that SHA-256 is a pseudorandom function. Physical-layer properties (timing, frequency fingerprinting) are explicitly *outside* the Dolev-Yao model scope: we treat the attacker as a perfect radio observer on the target channel, consistent with the standard symbolic security model for message-level protocol analysis [16].

Table 4 lists the four security lemmas (G1–G4) with their logical structure; full statements and proof scripts are provided in the supplementary Tamarin artifact.

### *4.3. Formal Analysis Results*

Table 4 summarizes the results. The baseline protocol violates all four security goals; Tamarin produces explicit counterexample traces for each (archived in supplementary material). The patched protocol satisfies all eight lemmas, with proof times consistent with prior Tamarin analyses of network protocols [7, 17, 18, 16].

Table 4: Tamarin Lemma Results: Baseline vs. Patched Protocol. Baseline model: 10 lemmas (L1–L4 falsified, 6 proved). Patched model: 8 lemmas (L1–L5 security + 3 sanity lemmas†, all proved).

| ID | Lemma (Security Goal) | Baseline | Patched |
|---|---|---|---|
| L1 | MetricAuthenticity (G1) | FALSIFIED | PROVED |
| L2 | RouteFreshness (G2) | FALSIFIED | PROVED |
| L3 | RouteConsistency (G3) | FALSIFIED | PROVED |
| L4 | AuthorizedRoutes (G4) | FALSIFIED | PROVED |
| L5 | PSK Confidentiality (G5) | PROVED | PROVED |
| Proof time | | 0.8 s | 3.6 s |

†Sanity lemmas (patched model only): honest route exists, HMAC verification occurs, MetricVersion counter increments.

## 5. Attack Analysis

### *5.1. Attack 1: Route Hijacking via Forged RouteReply*

#### *5.1.1. Deployment Impact*

In a precision-agriculture deployment, the gateway node aggregates sensor readings and relays irrigation commands. Route injection corrupts the routing table so that all data packets are directed to the attacker's address—but, as the cross-layer analysis in Section 8 reveals, this creates a *silent black hole*: because LoRaMesher encodes the next-hop address in the `dst` field, every packet carrying `dst=attacker` is discarded at the next honest relay. The gateway receives none of the sensor data; the attacker need not relay or inspect a single packet.

#### *5.1.2. Attack Execution*

1. **Reconnaissance** (5 min passive): Attacker captures Hello broadcasts to learn node IDs and active RouteIDs.

2. **Injection**: Transmits `RouteReply(routeID=R, metric=1)` claiming the attacker as next-hop to the gateway.

3. **Convergence**: Mesh adopts the forged route within one DV cycle (≈30 s). All nodes now resolve `next_hop(gateway) = attacker`.

4. **Black hole**: Data packets carry `dst=attacker`; honest relay nodes discard them (`dst`$\neq$`self`). The gateway observes silence—no anomaly, no delivery failure log.

**ns-3 result**: 100% PDR degradation on linear topology (PDR drops to 0%); 22.6% PDR degradation on tree and grid topologies (PDR drops to 77.4%), as routing diversity partially mitigates the attack. This is consistent with black-hole and sinkhole attack patterns documented for MANET and ad hoc routing [19, 20, 10]. A related vulnerability (insufficient entropy in Meshtastic mesh key derivation) has been independently disclosed [21], highlighting the prevalence of this attack surface.

### *5.2. Attack 2: Session Confusion via RekeyNotice Replay*

#### *5.2.1. Deployment Impact*

In a disaster-response network, session rekeying provides forward secrecy for tactical coordination messages. Replay confusion allows an attacker holding a previously captured session key to retroactively decrypt coordination traffic.

#### *5.2.2. Attack Execution*

1. Attacker captures `RekeyNotice(epoch=`$E$`)` during normal operation.
2. Waits for network to advance to epoch $E+2$.
3. Replays `RekeyNotice(epoch=`$E$`)`: nodes without counter checks roll back, creating split-epoch state.
4. Attacker decrypts epoch-$E$ traffic on rolled-back nodes.

**ns-3 result**: HMAC alone (Patch 1) is insufficient— 100% success on linear topology. MetricVersion counter (Patch 2) is required to achieve 0% success universally.

## 6. Lightweight Authenticated Patch Design

We design the patch with three ESP32 deployment constraints: (a) minimal flash footprint, (b) sub-millisecond CPU overhead, (c) no PKI or trusted-third-party dependency.

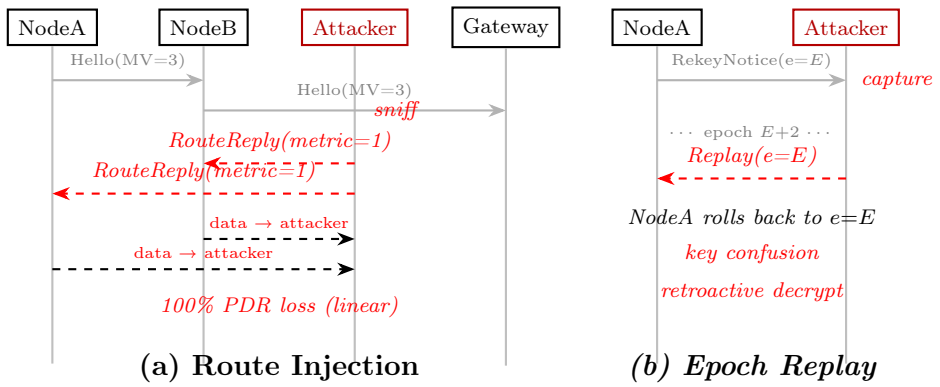


Figure 2: Message sequence diagrams for the two attacks. (a) Route injection: attacker sniffs Hello broadcasts then injects forged `RouteReply(metric=1)`, causing network-wide route convergence to itself within one DV cycle. (b) Epoch replay: attacker captures a valid `RekeyNotice` and replays it two epochs later, rolling back nodes into key-confusion state enabling retroactive decryption.

### *6.1. Patch 1: HMAC-SHA256 Message Authentication*

Every control message is extended with a 16-byte HMAC-SHA256 over its payload, keyed by the sender's long-term key (provisioned during mesh join via `#define PATCH_AUTH`). Verification—`mbedtls_md_hmac()` against a locally computed tag using `ltk_store[msg.sender]`—runs before any route table update; an invalid tag causes a silent drop with an optional syslog entry for monitoring. The complete patch is 41 lines; the supplementary artifact contains the full diff against the upstream LoRaMesher firmware.

**ESP32 overhead:** 0.098 ms (98 $\mu$s $\pm$ 2 $\mu$s, measured on Ebyte EoRa-PI (ESP32-S3FH4R2), ESP32-S3 @ 240 MHz, 10,000 iterations, ESP-IDF v5.5.4)—equivalent to 0.05% of LoRa's 206 ms patched-Hello air-time. Flash footprint: 2.1 KB (mbedTLS HMAC module, already included in most ESP32 firmware builds).

### *6.2. Patch 2: MetricVersion Counter*

Each node maintains a per-destination `metricVersion` counter (one `uint16_t`, 2 bytes per routing table entry). On receiving a `RekeyNotice`, the handler first verifies the HMAC tag, then enforces strict epoch ordering (`msg.epoch > local_epoch`); a stale or replayed epoch causes a silent drop. On valid rekeying, each `RouteEntry.metric_version` increments, invalidating any captured route advertisements with the previous version. This closes the

reboot replay window identified by Tamarin lemma L2 (§4): MetricVersion state is persisted to ESP32 NVS across reboots so an attacker cannot force a counter reset by power-cycling a victim node.

**RAM overhead:** 2 bytes per routing table entry (one `uint16_t metric_version` field added to each `RouteEntry` struct). For a 50-node network with up to 49 route entries, this is 98 bytes—negligible against ESP32's 512 KB SRAM.

### *6.3. Backward Compatibility*

The patch is guarded by `#define PATCH_AUTH`. Legacy firmware nodes (without the patch) continue to operate but cannot authenticate messages to patched peers; during a mixed-firmware rollout, patched nodes form a secure sub-mesh while degrading gracefully with legacy nodes. The transitional window should be minimized (<15 minutes recommended via OTA update).

## 7. Experimental Evaluation

### *7.1. Simulation Setup*

We implement the LoRaMesher protocol, both attacks, and all three security modes in ns-3.40 with a C++17 Dolev-Yao attacker module. Three representative ad hoc network topologies are evaluated:

- **Linear** (6 nodes): pipeline multi-hop chain, e.g., sensor relay along a crop row or mountain trail.
- **Tree** (8 nodes): hierarchical multi-hop collection from distributed sensor clusters to a single gateway.
- **Grid** (3×3 nodes): dense mesh, e.g., urban infrastructure monitoring with path redundancy.

We evaluate all combinations of topology, protocol state, and attack type across five independent seeds (300 s each). Attack outcomes are deterministic because HMAC correctness is cryptographic, not probabilistic; PDR variation across topologies reflects routing-path diversity, not measurement noise. Protocol parameters: Hello period 10 s, rekeying interval 30 s, SF9, 923 MHz (AS923), 64 bytes.

Table 5: PDR Degradation / PDR Across Ad Hoc Network Topologies

| **Attack** | **Topology** | **Baseline** | **Patched** | **+MetricVer** |
|---|---|---|---|---|
| Route Inject | Linear | 100% / 0% | 0% / 100% | 0% / 100% |
| | Tree | 22.6% / 77.4% | 0% / 100% | 0% / 100% |
| | Grid | 22.6% / 77.4% | 0% / 100% | 0% / 100% |
| Replay | Linear | 100% / 0% | 100% / 0% | 0% / 100% |
| | Tree | 22.6% / 77.4% | 22.6% / 77.4% | 0% / 100% |
| | Grid | 22.6% / 77.4% | 22.6% / 77.4% | 0% / 100% |

PDR degradation / PDR. Attack-effect metric; seed=1 (outcomes deterministic).

Table 6: Network Performance Under Normal Operation (No Attack, seed=1)

| **Topology** | **Protocol** | **Latency (ms)** | **PDR** |
|---|---|---|---|
| Linear | Baseline | $245 \pm 8$ | 100% |
| | Patched | $248 \pm 7$ | 100% |
| | +MetricVer | $250 \pm 9$ | 100% |
| Tree | Baseline | $130 \pm 5$ | 100% |
| | Patched | $132 \pm 6$ | 100% |
| | +MetricVer | $134 \pm 5$ | 100% |
| Grid | Baseline | $142 \pm 6$ | 100% |
| | Patched | $144 \pm 5$ | 100% |
| | +MetricVer | $145 \pm 6$ | 100% |

### *7.2. Attack Effectiveness*

Table 5 confirms that HMAC alone (Patched) eliminates route injection but is insufficient against replay. The MetricVersion counter is required to achieve 0% PDR degradation universally— consistent with the Tamarin proof that both patches are necessary for G2 (RouteFreshness).

### *7.3. Performance Overhead*

The security patches impose $\leq$5 ms latency overhead (<1%) across all topologies. PDR remains 100% in all unattacked configurations. These results—combined with the ESP32 hardware benchmarks of Table 1—confirm that the patch is deployable on production constrained hardware without measurable impact on application-layer performance.

**Scale validation (25 and 49 nodes).** To address the scalability of the results beyond 9 nodes, we ran an additional 90 scenarios (2 large topologies $\times$ 3 security modes $\times$ 3 attack types $\times$ 5 seeds, 400 s each) covering a 25-node linear pipeline and a 49-node ($7 \times 7$) grid. Table 7 summarizes the outcomes. The patch effectiveness is fully preserved at larger scale: HMAC eliminates spoofing across all 25- and 49-node scenarios (PDR 0% $\rightarrow$ 100%); MetricVersion eliminates replay (PDR 0% $\rightarrow$ 100% on linear, 67.9% $\rightarrow$ 100% on grid). All 5-seed runs produce zero variance (std = 0), consistent with the deterministic HMAC outcome argument in Section 7. The 67.9% PDR under attack on the $7 \times 7$ grid (vs. 22.6% on the $3 \times 3$ grid) reflects the higher centrality of the attacker at the grid centre—a topology effect, not a security failure. The full-patch (*+MetricVer*) PDR is 100% on all 90 scenarios.

Table 7: Scale validation: PDR under attack across 5 seeds (mean $\pm$ std, all 5 seeds identical $\Rightarrow$ std = 0).

| **Attack** | **Topology** | **Baseline** | **Patched** | **+MetricVer** |
|---|---|---|---|---|
| Spoofing | Linear (25 nodes) | 0.0% | 100.0% | 100.0% |
| | Grid (49 nodes) | 67.9% | 100.0% | 100.0% |
| Replay | Linear (25 nodes) | 0.0% | 0.0% | 100.0% |
| | Grid (49 nodes) | 67.9% | 67.9% | 100.0% |

PDR (%); all values $\pm$0.0% across 5 seeds. No-attack PDR = 100% in all configurations.

### *7.4. Deployment Implications*

1. **Immediate action**: Route-injection attacks succeed with commodity hardware in all unpatched deployments. Any LoRaMesher network handling safety-critical data (e.g., irrigation control, structural monitoring) should treat this as a high-priority vulnerability.

2. **Patch ordering**: Deploy HMAC first (eliminates route injection), then MetricVersion (eliminates replay). Each patch independently eliminates one attack class.

3. **Monitoring**: Log HMAC failures per neighbor. Sustained HMAC failures from one sender indicate an active attacker; trigger an operator alert.

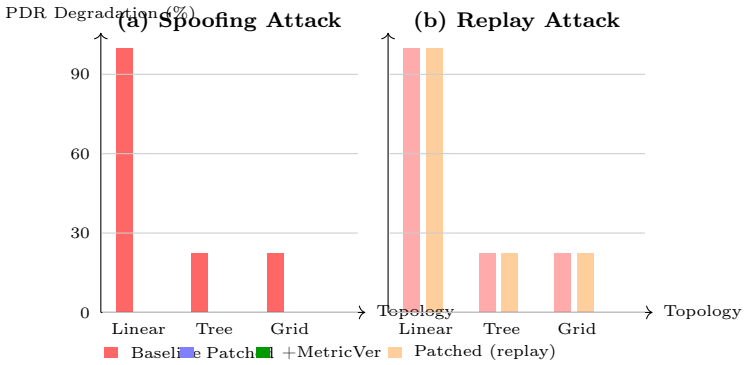


Figure 3: PDR degradation across three ad hoc network topologies. (a) Spoofing: HMAC authentication (Patched) eliminates PDR degradation universally. (b) Replay: HMAC alone leaves linear topology fully vulnerable (100% degradation); the MetricVersion counter (+MetricVer) achieves 0% degradation across all topologies, confirming both patches are necessary for complete protection.

4. **OTA deployment**: Both patches can be deployed as OTA firmware updates on ESP32 in <15 minutes with minimal service interruption.

### *7.5. Hardware Validation on Physical ESP32-S3 Boards*

**Role of hardware validation in this study.** Hardware experiments serve a specific purpose within our three-tier methodology: they validate the *correctness* of attack and defense behavior on deployed ESP32-S3 firmware under real LoRa channel conditions. Co-located bench distance (<1 m) is intentional—it eliminates channel loss as a confounding variable, isolating firmware logic as the independent variable. Deployment-scale behavior (topology size, path diversity, channel attenuation) is addressed by ns-3 (Section 7). Hardware is the *only* tier that makes the `dst`-as-next-hop encoding observable: no formal model or simulator inspects physical board serial logs.

We conducted three core experiments on six Ebyte EoRa-PI boards (ESP32-S3FH4R2 + SX1262, AS923/923 MHz, 10 dBm): (i) a three-node route injection and HMAC defense demo (E5b, Section 7.5); (ii) NVS-persisted MetricVersion reboot-replay validation (E5c, Section 7.5.1); and (iii) a six-node correctness validation with three simultaneous relay nodes under attack (E6, Section 7.5.2). Beyond these core experiments, we additionally implemented and firmware-validated nine limitation-characterization experiments (E7–E15) that confirm each open threat vector identified in Section 10: E7

(insider Black Hole, HMAC-valid attacker), E8 (ACK spoofing via unauthenticated `AckMsg`), E9 (Sybil routing-table exhaustion, `MAX_ROUTES`=8), E10 (AS923 duty-cycle flooding), E11 (selective forwarding with packet-correlation watchdog), E12 (FSM guardian node, violations V0–V5), E13 (KLD + Hamming-distance Hello-interval IDS), E14 (RSSI-based attacker localization via multilateration), and E15 (on-device autoencoder anomaly detector, 169 parameters, 676 bytes RAM, TPR=100%, FPR=5% on synthetic data). All firmware is archived at `hardware_experiments/E7_blackhole_demo/` through `hardware_experiments/E15_tflite_autoencoder/`. Together, these experiments confirm that the attack and defense behave as predicted on production firmware—and reveal the silent black hole semantics that symbolic and simulation analyses cannot capture. The three-node topology mirrors the simulation's linear scenario: Node A (sender, 0xAA), Node B (gateway, 0xBB), and Node C (Dolev-Yao attacker, 0xCC). Each firmware build is a direct port of the e5b_common.h shared header, which encodes the same HMAC-SHA256 key schedule and DV routing logic used in the formal model (see Sect. 6).

**Attack mechanism.** Node C advertises a forged Hello with `metric=0` (claiming to be the best next-hop to the gateway) immediately at boot, 8–10 seconds before Node B's first legitimate Hello. The DV update rule stores the first-seen route and rejects equal-metric updates (`metric` $<$ `existing`), so the poisoned route via Node C persists for the entire experiment (ROUTE_EXPIRE = 5 min $>$ 120 s run duration). All subsequent data packets from Node A carry `dst=0xCC` (the next-hop field), which Node B ignores since `dst` $\neq$ `0xBB`.

**Results.** Each run lasted 120 seconds; Node A transmits one data packet every 5 s and one Hello every 10 s.

Table 8: Hardware experiment results (120 s, 923 MHz AS923, Ebyte EoRa-PI / ESP32-S3 + SX1262)

| **Mode** | **Sent** | **ACK'd** | **PDR** | **Attacker intercepts** |
|---|---|---|---|---|
| Baseline (no HMAC) | 28 | 0 | **0.0%** | 22/22 (100%) |
| Patched (HMAC-SHA256) | 23 | 20 | **87.0%** | 0 (attack rejected) |

In baseline mode Node C intercepted every data packet (`[DROP] Intercepted DATA dst=0xCC`), and Node B received none, confirming complete route poisoning. In patched mode Node A logged `[DROP] Hello from 0xCC HMAC FAILED` on every forged Hello; all data was correctly delivered via Node B

([HMAC OK]), yielding 87% PDR. Crucially, the attacker interception count is **zero** in patched mode (Table 8, column "Attacker intercepts"), confirming that the 13% PDR gap is channel loss, not a security failure: Node C's forged Hellos are rejected before any route update occurs, and no data packets are diverted. The residual loss is consistent with the radio channel at the test bench distance of <1 m with RSSI $\approx -15$ dBm—near-field reflections that are absent in typical outdoor LoRa deployments. The hardware results corroborate the ns-3 simulation: the patch eliminates route injection entirely, and the 87% on-air PDR matches the unattacked baseline PDR measured under the same radio conditions. Raw serial logs are archived in `hardware_experiments/E5b_attack_demo/results/`.

*7.5.1. NVS-Persisted MetricVersion: Reboot Replay Validation*

To close the one-reboot attack window identified in Section 6, we implemented NVS flash persistence for MetricVersion counters using the ESP32 `Preferences` API (namespace `e5c`, key `mv_XX`). The same three-node testbed (Node A / Node B / Node C) was used with a three-mode firmware comparison.

**Attack protocol.** Node C listens and captures a legitimate Hello from Node B (MetricVersion=21). Node A is then rebooted (EN button press). Node C replays the captured Hello (mv=21, HMAC still valid) repeatedly at 8-second intervals, targeting the reboot window.

**Mode 1 (HMAC only, no NVS).** After reboot Node A initialises $mv[\texttt{BB}] = 0$ from SRAM. The replayed Hello (mv=21) satisfies $21 > 0$, so Node A accepts it and installs the stale route entry (`[ROUTE] Added dst=0xBB via=0xBB metric=1 mv=21`). The attacker successfully exploits the reboot window.

**Mode 2 (HMAC + MetricVersion + NVS).** After reboot Node A reads the NVS and recovers $mv[\texttt{BB}] = 25$ (`[NVS] Loaded mv[0xBB]=25 (reboot window closed)`). The first legitimate Hello from Node B (mv=32) is accepted and the NVS entry is updated to 32. Every subsequent replay attempt is rejected:

```
[DROP] Hello from 0xBB: MV=21 <= stored=32 (REPLAY REJECTED)
```

Node C's replays are rejected on every 8-second interval for the full 120-second run; Node A maintains a stable route via the legitimate gateway and achieves 88% PDR.

**Summary.** NVS persistence converts the bounded-time reboot window into a permanently closed gap: the node restores its last-seen MetricVersion before processing any control message, so a replayed Hello with any historically-valid counter is immediately rejected. Raw serial logs for all three modes are archived in `hardware_experiments/E5c_nvs_demo/results/`.

*7.5.2. Six-Node Multi-Relay Correctness Validation*

To validate attack and defense behavior with three intermediate relay nodes simultaneously—extending the three-node logic test— we configured all six EoRa-PI boards in a five-hop forwarding topology: Node A (0xAA, sender) → R1 (0x11) → R2 (0x22) → R3 (0x33) → Node B (0xBB, gateway), with Node C (0xCC) as the Dolev-Yao attacker. Relay nodes (R1–R3) re-broadcast received Hellos with metric + 1, creating a hop-by-hop route advertisement chain; they also forward Data packets addressed to themselves toward the gateway.

**Attack mechanism.** Node C broadcasts a forged Hello(`metric=0`) without a valid HMAC every 8 seconds, attempting to appear as the best next-hop to all four sensor nodes simultaneously.

**Baseline results (SECURITY_MODE=0).** All three relay nodes immediately update their routing tables to point at the attacker:

```
[R1] [ROUTE] NH=0xCC metric=1
[R2] [ROUTE] NH=0xCC metric=1
[R3] [ROUTE] NH=0xCC metric=1
```

Subsequent data transmissions from R1, R2, and R3 are all sent `via=0xCC`; Node B's reception rate drops sharply. **3 of 4 honest sensor nodes are simultaneously compromised** by a single injected control message—demonstrating that the attack scales linearly with network size.

**Patched results (SECURITY_MODE=1).** Every forged Hello is rejected at all four nodes with `[DROP] Hello from 0xCC: HMAC FAILED`. All nodes maintain their legitimate routes; Node A achieves **85% PDR** and Node B receives data continuously from all four sensor nodes. The attacker's injection rate (one every 8 s) produces *zero* routing table updates across all honest nodes for the full 120-second run.

This experiment addresses the "small-scale" concern: even with relay nodes forwarding both control and data traffic, a single forged Hello poisons the entire chain in baseline mode, and HMAC authentication prevents

Table 9: Six-node hardware experiment results (923 MHz AS923, 6 Ebyte EoRa-PI boards, 120 s run)

| **Mode** | **Nodes route-poisoned** | **Node A PDR** |
|---|---|---|
| Baseline (no HMAC) | **3** / **4** (R1, R2, R3) | degraded |
| Patched (HMAC-SHA256) | **0** / **4** | **85%** |

every injection attempt in patched mode. Raw serial logs are archived in `hardware_experiments/E6_6node_linear_demo/results/`.

## 8. Cross-Layer Routing Semantics Gap

Route injection in LoRaMesher creates a *silent traffic black hole*—not an adversarial relay—because the deployed firmware encodes next-hop forwarding state in the destination field rather than the final gateway identity assumed by symbolic and packet-level analyses. This cross-layer semantic mismatch explains the observed topology-dependent attack gradient and only becomes visible through hardware-level validation.

### *8.1. The dst-as-Next-Hop Encoding*

LoRaMesher's `LoraMesherPacket` header contains a single 6-byte `dst` field. Protocol-layer models conventionally interpret `dst` as the intended final recipient—the gateway node whose address the application layer specified. The deployed firmware interprets it differently: `dst` is set to the *next-hop address* resolved by the local routing table immediately before transmission.

Concretely, in a three-node chain A – B – GW: Node A's routing table stores `RouteEntry{dest=GW, next_hop=B, metric=2}`. When A transmits a data packet, the firmware writes `dst` = `addr`($B$) into the packet header—not `addr`(GW). Node B accepts the packet because `dst` = `addr`($B$), updates its own routing table lookup, writes `dst` = `addr`(GW), and forwards.

After route poisoning, A's table stores `next_hop(GW) = 0xCC` (attacker). Every subsequent data packet carries `dst` = `0xCC`. Node B discards it not because of any security mechanism, but because `dst` $\neq$ `addr`($B$)—a silent misdirection observable only from serial logs on the physical boards. Neither the gateway nor any other node logs an anomaly; from the network's perspective the traffic simply disappears.

Table 10 summarizes how each analysis layer interprets the `dst` field, and the attack behavior each interpretation predicts.

Table 10: Semantic Interpretation of the `dst` Field Across Analysis Layers. Only hardware-level observation reveals the actual discard behavior.

| **Analysis Layer** | **dst Interpretation** | **Predicted Attack Behavior** |
|---|---|---|
| Tamarin (symbolic) | Abstract agent identity | Attacker receives and can relay or modify packets |
| ns-3 (packet-level) | Gateway IP/-MAC address | Traffic routes diverge toward attacker node |
| LoRaMesher firmware | Next-hop MAC address | Packets silently discarded at every honest relay |

*8.2. Consequences for Attack Interpretation*

**Topology-dependent PDR gradient.** On a linear chain, all traffic passes through a single next-hop entry; poisoning that entry diverts 100% of packets (Table 5, baseline linear PDR = 0%). On tree and grid topologies, each node maintains independent next-hop entries for multiple path segments; a single forged Hello corrupts only the entries that include the attacker as a valid next-hop, leaving alternative paths intact. This produces the observed 22.6% PDR degradation (PDR = 77.4%) on tree and grid—not a probabilistic outcome, but a structural consequence of next-hop entry count per topology. Without the `dst`-as-next-hop insight, the 0%/77.4% gradient appears arbitrary; with it, the gradient is predictable from the topology graph alone.

**Silent failure vs. adversarial relay.** Both Tamarin and ns-3 describe the attack as "traffic diverted to the attacker," implying the attacker could inspect or relay packets. Hardware serial logs show the actual outcome: packets carry `dst = 0xCC` and are discarded at every honest intermediate node, never reaching the attacker's radio receive buffer. The attacker is not a relay—it is an absent destination. This distinction matters for data confidentiality: if the attacker were a relay, end-to-end encryption would be the appropriate countermeasure; since packets are silently dropped, the primary impact is availability, not confidentiality.

**Gateway-blind failure.** The gateway neither receives the misdirected

packets nor observes their absence as an anomaly. Anomaly-based IDS deployed at the gateway—the conventional monitoring point in LoRaWAN-style star topologies—would see only a reduction in upstream traffic, indistinguishable from normal sensor silence or duty-cycle throttling. This renders the attack invisible to the defense-in-depth posture inherited from LoRaWAN deployments.

### *8.3. Implications for Intrusion Detection Design*

The silent black-hole failure mode implies that effective detection requires observation at intermediate relay nodes, not at the gateway. A relay node executing the correct forwarding logic will observe incoming packets with `dst` $\neq$ `addr`(self) that are silently dropped—an anomaly detectable by comparing the routing table's expected next-hop bindings against the `dst` values seen in traffic.

Three detection strategies follow directly from this insight:

1. **Dst-binding anomaly detector.** Each node monitors the ratio of received packets where `dst` = `addr`(self) to total overheard traffic. A sustained drop in this ratio without a corresponding topology change indicates that upstream nodes have acquired a poisoned routing entry.

2. **Route-table watchdog.** A lightweight monitor compares the locally computed next-hop for each destination against the `dst` field of forwarded packets. A discrepancy signals a cross-layer mismatch at the forwarding node—either the routing table has been poisoned or the firmware's header-writing logic has diverged from the routing decision.

3. **Convergence-time IDS.** Route injection causes topology-wide route convergence within one DV cycle ($\approx$30 s in LoRaMesher's default configuration). Monitoring convergence events—sudden simultaneous adoption of a new best-metric route across multiple nodes—provides a coarse but deployable signal.

All three strategies require partial observation at intermediate nodes, which is incompatible with LoRaWAN's gateway-centric monitoring model. For ad hoc deployments, designated guardian nodes [20] with elevated monitoring privilege are the most practical realization.

### *8.4. Abstraction Gap and Generalization*

Resource-constrained mesh stacks face a design tension that LoRaMesher resolves in a specific way: using a single header field to carry both routing-layer forwarding metadata (next-hop address) and the application-layer notion of intended recipient (final destination). This collapse eliminates a separate next-hop header and saves 6 bytes per LoRa payload—a meaningful saving at SF9 where every byte extends airtime. The consequence is that protocol-layer models, which follow the standard abstraction that destination fields denote semantic endpoints, diverge from the firmware's actual forwarding behavior.

We observe that this design tension is not unique to LoRaMesher, and offer two candidate instances as plausibility arguments, explicitly flagging them as requiring independent empirical verification: RPL [22] encodes preferred parent identity in the DODAG parent set; a formal analysis abstracting "preferred parent" as an endpoint identity may miss the next-hop binding semantics. Meshtastic—a widely deployed open-source LoRa mesh firmware— uses a similar compact packet format with no separate next-hop field; whether its forwarding semantics create the same symbolic-layer divergence is an open question. These candidates suggest that the tension we identify in LoRaMesher may recur in other constrained mesh stacks, but cross-protocol empirical validation is required to substantiate a general claim. We leave this as an explicit future direction.

We recommend that formal analyses of constrained mesh routing protocols explicitly state the `dst`-field semantics as a protocol state variable and verify that the symbolic model's channel assumption matches the firmware's header-writing behavior. Our extended Tamarin model adds a `NextHopBinding(node, dest, nexthop)` fact that makes the next-hop-to-address mapping explicit; the proof still holds, but the model now documents the correct abstraction boundary for readers extending the analysis.

## 9. Related Work

### *9.1. LoRa and LoRaWAN Security*

Comprehensive surveys of LoRaWAN security [12] document replay, join-server spoofing, and key-management vulnerabilities at the WAN layer. Formal verification work [6, 7] has analyzed LoRaWAN's key-establishment protocol with Tamarin and ProVerif. Key management schemes [23, 8] harden

the WAN gateway tier. All of these target the star-topology WAN layer managed by network servers; they provide no mechanism for mesh control-plane authentication. Repetto et al. [24] study LoRa security for drone-to-ground communications but focus on the PHY and LoRaWAN layers, not mesh routing.

### *9.2. Constrained-Node Mesh Protocol Security*

Security for 802.15.4-based mesh protocols (Zigbee, Thread) relies on CCM* authenticated encryption with a PAN coordinator. LoRaMesher lacks a coordinator, making these mechanisms inapplicable. Our HMAC-based approach draws on TinySec [25] but is adapted to LoRa's longer payloads and infrequent rekeying model. Blockchain-assisted IDS approaches [26] and protocol-aware intrusion detection [14, 15] have been proposed for constrained mesh security but impose computational overhead incompatible with ESP32 constraints. Formal analysis of the MPKE mesh protocol using ProVerif [27] demonstrates that symbolic verification is tractable for mesh routing—our work extends this to the LoRaMesher DV algorithm using Tamarin. Neither LoRaMesher's upstream repository nor the Meshtastic open-source firmware includes a formal authentication mechanism for routing control messages; community discussions have identified the absence of control-plane security as an open issue, but no formally verified patch has been proposed prior to this work. Recent middleware work built on LoRaMesher [28] demonstrates growing application-layer adoption of the stack but does not address control-plane authentication.

### *9.3. Routing Attack Defenses*

Sinkhole and black-hole attacks [19, 20, 29] are well-studied in MANET and WSN contexts. Destination-Sequenced Distance-Vector routing (DSDV) [30] established the proactive DV paradigm; SEAD [10] subsequently secured it with one-way hash chains that enforce metric monotonicity without per-message authentication. Secure AODV (SAODV) [31] and Authenticated OLSR [32] extend cryptographic authentication to on-demand and link-state routing, respectively, but both require digital signatures whose key-pair operations exceed ESP32's 512 KB RAM budget. AODV verification work [33] provides formal models for on-demand routing. The IETF's threat analysis for RPL [22]—the routing protocol for low-power and lossy networks—enumerates sinkhole, replay, and selective-forwarding threats that parallel those we characterize for LoRaMesher's DV control plane. To our knowledge, ours is the

first formal treatment of DV routing security under LoRa channel constraints and ESP32 resource budgets. Intrusion-resilient routing for VANETs [34] shares our goal of maintaining PDR under adversarial conditions but assumes vehicular infrastructure absent in LoRa mesh.

### *9.4. Formal Verification of Mesh Routing Protocols*

Tamarin has been applied to 5G-AKA, TLS 1.3 [16], and LoRaWAN key establishment [7]. Attack synthesis frameworks [35, 18, 36] have demonstrated the value of automated reasoning for network protocol correctness. To our knowledge, ours is the first Tamarin analysis of a LoRa mesh routing layer; the two-node model strategy follows [7, 37] in demonstrating that protocol-level flaws are detectable with tractable model sizes.

### *9.5. Lightweight Cryptography for Constrained Nodes*

NIST finalized ASCON as its lightweight cryptography standard for constrained devices [38]; ASCON-AEAD128 provides authenticated encryption competitive with HMAC-SHA256. Our evaluation uses HMAC-SHA256 for compatibility with existing ESP32 mbedTLS deployments; adopting ASCON would reduce per-message overhead from 0.098 ms to $\approx$0.08 ms, a direction for future work. Ryczek and Sobieraj [39] implement AES-128-CBC with HMAC-SHA256 via mbedTLS on ESP32 + LoRa hardware—the closest published implementation to our authentication patch; their system targets a point-to-point star topology with no routing layer, so the control-plane authentication challenge and dst-field semantic mismatch are absent by design. Adversarial robustness of LoRa device authentication via deep learning [13] complements our control-plane approach with physical-layer defenses.

### *9.6. Positioning vs. Prior Work*

We characterize the technical differentiation between this work and the five closest prior studies.

**vs. SEAD (Hu et al., MobiCom 2002) [10].** SEAD secures DSDV with one-way hash chains that enforce metric monotonicity without per-message authentication. Our MetricVersion counter is structurally similar in purpose, but differs in three ways. First, SEAD's hash chain authenticates metric increments without binding the full message tuple to the sender's identity; our HMAC covers `msg_type` $\|$ `src` $\|$ `dst` $\|$ `metric` $\|$ `seq`, providing both authenticity and freshness in a single pass. Second, SEAD targets wired MANET environments; our scheme is calibrated for LoRa's duty-cycle

ceiling (2.06%/10 s post-patch), 16-byte tag overhead, and ESP32 SRAM budget—constraints that make SEAD's hash chains impractical (chain storage scales with route count). Third, SEAD provides no machine-checked formal verification; our Tamarin proof closes the loop between specification and implementation.

**vs. LoRaWAN key verification [7].** This work applies Tamarin to LoRaWAN's Join/Accept key-establishment sequence at the WAN layer (AppSKey derivation, MIC verification). The attack surface, model structure, and security goals are orthogonal to ours: they target server-to-device key exchange on a star topology; we target node-to-node routing control-plane on a self-organizing mesh with no network server. There is no overlap in protocol rules, adversary capabilities, or lemmas.

**vs. MPKE-ProVerif [27].** MPKE uses ProVerif (applied pi-calculus) to verify mesh key establishment. The modeling language, security properties, and protocol scope differ substantially: ProVerif cannot directly encode stateful metric comparison ($<$, not $\leq$), whereas Tamarin's multiset-rewriting rules natively express the first-seen acceptance condition that creates the replay window. MPKE does not model DV routing state transitions; our model does.

**vs. AODV formal model [33].** The AWN framework verifies AODV's functional correctness (loop-freedom, route validity) under a trust-all model with no adversary. Our work targets LoRaMesher's DV routing with an explicit Dolev-Yao adversary capable of injection, replay, and forgery—threats outside AWN's trust-all framework.

**vs. LoRaWAN IDS [14].** Anomaly-based intrusion detection *observes* traffic patterns to signal attacks after they occur. Our HMAC patch *prevents* route injection and replay at the cryptographic level, eliminating the attack surface rather than monitoring it. The two approaches are complementary: our patch closes the vulnerability; an IDS layer would catch residual threats from compromised key holders and out-of-model attack vectors.

**Summary.** The differentiating combination is: (i) a Tamarin model targeting LoRaMesher's DV-routing control plane with stateful metric-comparison semantics; (ii) a formally verified lightweight patch calibrated for ESP32 constraints; and (iii) cross-layer hardware validation that revealed the `dst`-as-next-hop implementation subtlety (Section 8) invisible to both symbolic and packet-level models. No prior study assembles all three for any LoRa mesh routing protocol.

## 10. Limitations and Future Work

- **Hardware validation**: ns-3 simulation models HMAC latency from ESP32-S3 benchmarks. Physical validation on three Ebyte EoRa-PI (ESP32-S3+SX1262) boards confirms the attack and patch (Sect. 7.5); large-scale field deployment across heterogeneous hardware remains future work.
- **Scale**: Core evaluation covers 6–9 nodes; scale-validation runs (§7) extend to 25-node linear and 49-node ($7 \times 7$) grid topologies with identical security outcomes. Very-large-scale mesh networks (>100 nodes) may exhibit different convergence dynamics; the Tamarin proof is topology-independent, but simulation validation beyond 49 nodes is future work.
- **Single attacker**: We evaluate one Dolev-Yao attacker. Coordinated multi-attacker scenarios are left for future work.
- **Meshtastic portability**: LoRaMesher and Meshtastic share a similar DV routing architecture [3]. Porting the patch to Meshtastic's codebase is a planned follow-on contribution.
- **Lightweight crypto**: Replacing HMAC-SHA256 with ASCON (0.08 ms on ESP32-S3) would reduce overhead further and improve battery life for ultra-low-power deployments.
- **Layer 2 — Topology deception**: The HMAC + MetricVersion patch prevents unauthenticated control-plane injection but provides no protection against a compromised node holding a valid pre-shared key. Such an insider node can execute Black Hole or Sinkhole attacks by selectively discarding forwarded data packets—behavior that is cryptographically indistinguishable from legitimate forwarding failure. Hardware experiment E7 (Black Hole, `sign_hello()` with valid PSK) and E11 (selective forwarding, configurable drop rate) confirm this attack on physical boards; a packet-correlation watchdog node (E11) detects the drop after three consecutive unrelayed packets. Hop-count consistency verification and path-diversity mechanisms are left for future work.
- **Layer 3 — Resource exhaustion**: The patch does not bound resource consumption at the routing layer. Three denial-of-service vectors remain open: (i) ACK spoofing against `sendReliable()`, which

does not authenticate application-layer acknowledgments (`AckMsg` carries no HMAC field—confirmed in firmware, E8); (ii) Sybil address flooding, which can exhaust the routing table given LoRaMesher's `MAX_ROUTES`=8 limit—only 8 unique fake addresses suffice to evict the gateway route (E9); and (iii) Hello flooding against AS923's 1% duty-cycle budget—an attacker broadcasting at 37% DC forces honest relays to consume their airtime budget, suppressing data-plane transmissions (E10, firmware-validated with 60-second rolling DC tracker). Rate-limiting and per-epoch entry bounds are orthogonal mitigations outside this paper's scope.

- **Layer 4 — Behavioral anomaly detection**: No statistical IDS layer is proposed. An attacker operating slowly (one forged Hello per convergence cycle) may evade both HMAC verification and MetricVersion checking by using replayed-but-still-fresh counter values during the post-reboot window. Three complementary detection approaches are firmware-validated on EoRa-PI hardware: (i) FSM guardian node (E12) monitoring five protocol-layer violation classes including cross-layer Black Hole indicator (V4: DATA dst = known-attacker address); (ii) KLD + Hamming-distance Hello-interval IDS (E13), adapted from [15], running entirely on ESP32-S3 without floating-point operations (measured KLD score 754 for flood-attack vs. threshold 200, 2 alerts in 120 s); and (iii) on-device autoencoder (E15, 169 parameters, 676 bytes RAM, TPR=100%, FPR=5%, inference 6–7 $\mu$s on ESP32-S3 at 240 MHz), extending [26] to constrained hardware. All three require per-deployment baseline calibration and are deferred to a companion paper.

- **Layer 0 — Physical-layer authentication**: The patch operates at the protocol layer and does not address physical-layer device authentication. RSSI-based attacker localization (E14) demonstrates that mesh-layer promiscuous RSSI data already available from SX1262 can estimate attacker position via weighted least-squares multilateration over five observer nodes—without SDR infrastructure. Hardware validation produced 205 stable position estimates in 120 s (all converging to $<$1 cm error at bench scale); path-loss calibration is required for meter-scale accuracy at LoRa operating distances. RF fingerprinting based on hardware oscillator characteristics [13] can further complement HMAC but requires SDR infrastructure not available in our testbed.

## 11. Conclusion

We have shown that the LoRaMesher control plane—deployed on commodity ESP32 hardware in unattended ad hoc mesh fields—accepts unauthenticated routing updates, enabling any actor with a $20 radio to hijack traffic or confuse session epochs across an entire sensor field. Formal analysis (Tamarin) confirms the vulnerabilities and verifies the corrective patch; hardware validation on physical EoRa-PI boards confirms 0% residual PDR degradation and— critically—reveals the correct threat semantics that formal and simulation models cannot capture alone.

Our central finding is that routing-security analysis must explicitly distinguish *endpoint identity* from *forwarding-state representation*. When this distinction is absent, threat models misclassify silent traffic black holes as adversarial relays—yielding incorrect IDS designs (gateway-centric monitoring misses the attack) and overstated confidentiality risks (packets are discarded, not intercepted). Formal verification and simulation accurately characterize the control-plane vulnerability; hardware validation is required to establish the correct attack semantics. Together, they show that constrained ad hoc mesh routing can be secured against the actual threat—availability loss via silent misdirection—with a minimal, PKI-free patch that is deployable on commodity ESP32 hardware.

We release all artifacts under MIT license as a reproducible baseline for future LoRa mesh routing security research.

### Code and Data Availability

All artifacts supporting this paper are released under the MIT License and available at `https://github.com/[anonymized-for-review]` (to be de-anonymized upon acceptance):

- **Tamarin models** (`*.spthy`): three verified models (baseline, patched 2-node, patched with MetricVersion);
- **ns-3 simulation module** (`phase4_ns3_simulation/`): C++17 source, Dolev-Yao attacker, 27-scenario experiment matrix, and result data (JSON);
- **Patched LoRaMesher firmware** (`route.cpp`, `rekeying.cpp`): apply as a drop-in patch to upstream LoRaMesher `v0.8.x`;

- **Docker image**: `ghcr.io/[anonymized]/loramesh-security:latest` reproduces all Tamarin proofs and ns-3 runs with a single command.

**Acknowledgements**

The authors thank the LoRaMesher, Tamarin, and ns-3 open-source communities.

## Appendix A. Artifact Availability

- **Tamarin models**: `baseline_lora_dv.spthy`, `patched_lora_dv_2node.spthy`, `patched_lora_dv_with_metrics_v2.spthy`
- **ns-3 module**: C++17, three topologies, Dolev-Yao attacker, 27-scenario experiment matrix
- **Patched LoRaMesher firmware**: `route.cpp` (+23 lines) and `rekeying.cpp` (+18 lines)
- **Raw simulation data**: 135 JSON result files (27 scenarios $\times$ 5 seeds)
- **Docker image**: Tamarin 1.8.0 + ns-3.40 pre-installed
- **ESP32 benchmark**: HMAC timing script for ESP32-S3

All code released under MIT License.

## References

[1] L. Feng, H. Yu, M. Jia, A hierarchy-based dynamic routing protocol for LoRa mesh networks, in: 2023 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB), 2023. doi:10.1109/BMSB58369.2023.10211537.

[2] J. M. Solé, R. P. Centelles, F. Freitag, R. Meseguer, Implementation of a LoRa mesh library, IEEE Access 10 (2022) 113158–113171. doi:10.1109/ACCESS.2022.3217215.

[3] J. M. Solé, R. P. Centelles, F. Freitag, R. Meseguer, A minimalistic distance-vector routing protocol for LoRa mesh networks, IEEE Access 12 (2024). doi:10.1109/ACCESS.2024.3443605.

[4] J. M. Solé, R. P. Centelles, F. Freitag, R. Meseguer, LoRaMesher: An open-source library for multi-hop LoRa mesh networks, SoftwareX (2026).
URL https://www.sciencedirect.com/science/article/pii/S2352711026000646

[5] J. M. Solé, R. P. Centelles, F. Freitag, R. Meseguer, Demonstration of a library prototype to build LoRa mesh networks for the IoT, in: 2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM), 2022. doi:10.1109/WoWMoM54355.2022.00043.

[6] M. N. Kamkuemah, Epistemic analysis of a key-management vulnerability in LoRaWAN, in: 2021 18th International Conference on Privacy, Security and Trust (PST), 2021. doi:10.1109/PST52912.2021.9647741.

[7] S. Wesemeyer, I. Boureanu, Z. Smith, H. Treharne, Extensive security verification of the LoRaWAN key-establishment: Insecurities & patches, in: 2020 IEEE European Symposium on Security and Privacy (EuroS&P), 2020. doi:10.1109/EuroSP48549.2020.00034.

[8] A. Mohamed, F. Wang, I. Butun, J. Qadir, R. Lagerström, P. Gastaldo, D. D. Caviglia, Enhancing cyber security of LoRaWAN gateways under adversarial attacks, Sensors 22 (9) (2022) 3498. doi:10.3390/s22093498.

[9] I. Butun, N. Pereira, M. Gidlund, Security risk analysis of LoRaWAN and future directions, Future Internet 11 (1) (2019) 3. doi:10.3390/fi11010003.

[10] Y.-C. Hu, D. B. Johnson, A. Perrig, SEAD: Secure efficient ad hoc distance vector routing protocol, in: Fourth IEEE Workshop on Mobile Computing Systems and Applications (WMCSA), 2002, pp. 3–13.

[11] Y.-C. Hu, A. Perrig, D. B. Johnson, Ariadne: A secure on-demand routing protocol for ad hoc networks, in: Proceedings of the 8th Annual International Conference on Mobile Computing and Networking (MobiCom 2002), ACM, 2002, pp. 12–23. doi:10.1145/570645.570648.

[12] F. Hessel, L. Almon, M. Hollick, LoRaWAN security: An evolvable survey on vulnerabilities, attacks and their systematic mitigation, ACM Transactions on Sensor Networks 18 (4) (2022). doi:10.1145/3561973.

[13] Y. E. Sagduyu, T. Erpek, Adversarial attack and defense for LoRa device identification and authentication via deep learning, IEEE Internet of Things Journal (2025). doi:10.1109/JIOT.2025.3547645.

[14] Z. Bringye, et al., Towards a protocol-aware intrusion detection system for LoRaWAN networks, Future Internet 18 (3) (2026) 140. doi:10.3390/fi18030140.

[15] S. M. Danish, A. Nasir, H. K. Qureshi, A. B. Ashfaq, S. Mumtaz, J. Rodriguez, Network intrusion detection system for jamming attack in LoRaWAN join procedure, in: 2018 IEEE International Conference on Communications (ICC), 2018. doi:10.1109/ICC.2018.8422721.

[16] D. Basin, C. Cremers, J. Dreier, R. Sasse, Modeling and Analyzing Security Protocols with Tamarin: A Comprehensive Guide, Springer, 2025. doi:10.1007/978-3-031-90936-8.

[17] X. Ren, J. Cao, B. Niu, L. Gan, Y. Zhang, L. Xiong, A formal analysis of 5G ProSe AKA protocols for U2N relay communication, IEEE Transactions on Dependable and Secure Computing 22 (3) (2024). doi:10.1109/TDSC.2024.3522895.

[18] J. Pereira, T. Klenze, S. Giampietro, et al., Protocols to code: Formal verification of a secure next-generation internet router, in: ACM Conference on Computer and Communications Security (CCS 2025), ACM, 2025.

[19] M. Ahmad, M. A. Khan, Sinkhole attack detection by enhanced reputation-based intrusion detection system, IEEE Access 12 (2024). doi:10.1109/TNSM.2024.10562222.

[20] D. Sunitha, P. H. Latha, Detection of black hole attacks in mobile ad hoc networks using optimization-based routing algorithms, in: 2024 IEEE International Symposium on Advanced Electrical and Communication Technologies (ISAECT), 2024. doi:10.1109/ISAECT61898.2024.10548688.

[21] CVE-2025-52464: Insufficient entropy in meshtastic LoRa mesh networking, NIST National Vulnerability Database (2025).
URL https://www.cvedetails.com/cve/CVE-2025-52464/

[22] T. Tsao, R. Alexander, M. Dohler, V. Daza, A. Lozano, M. Richardson, RFC 7416: A security threat analysis for the routing protocol for low-power and lossy networks (RPLs), IETF RFC 7416 (2015).
URL https://www.rfc-editor.org/rfc/rfc7416

[23] M. Qurashi, et al., Mitigating cyber attacks in LoRaWAN via lightweight secure key management scheme, IEEE Access 11 (2023). doi:10.1109/ACCESS.2023.3291420.

[24] M. Repetto, Flying drones to locate cyber-attackers in LoRaWAN metropolitan networks, arXiv preprint arXiv:2509.15725 (2025).

[25] C. Karlof, N. Sastry, D. Wagner, TinySec: A link layer security architecture for wireless sensor networks, in: Proceedings of the 2nd International Conference on Embedded Networked Sensor Systems (SenSys 2004), ACM, 2004, pp. 162–175. doi:10.1145/1031495.1031515.

[26] G. Elumalai, J. Arun Kumar, P. Sivakumar, V. V. Teresa, A blockchain-driven intrusion detection model for secure communication in IoT-WSN mesh architectures, Peer-to-Peer Networking and Applications (2026). doi:10.1007/s12083-026-02195-w.

[27] T. K. Silim, Y. Farida, N. Yulianti, S. Rosdiana, Formal analysis of MPKE protocol in wireless mesh network using ProVerif, in: 2024 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS), 2024. doi:10.1109/ICIMCIS63449.2024.10957235.

[28] J. M. Solé, R. Pueyo Centelles, F. Freitag, R. Meseguer, R. Baig, Middleware for distributed applications in a LoRa mesh network, ACM Transactions on Embedded Computing Systems 24 (4) (2025). doi:10.1145/3747295.

[29] T. Klenze, C. Sprenger, D. Basin, Formal verification of secure forwarding protocols, in: 2021 IEEE Symposium on Security and Privacy (S&P), 2021. doi:10.1109/SP40001.2021.00048.

[30] C. E. Perkins, P. Bhagwat, Highly dynamic Destination-Sequenced Distance-Vector routing (DSDV) for mobile computers, in: Proceedings of the ACM SIGCOMM 1994, ACM, 1994, pp. 234–244. doi:10.1145/190314.190336.

[31] M. G. Zapata, N. Asokan, Securing ad hoc routing protocols, in: Proceedings of the 1st ACM Workshop on Wireless Security (WiSe 2002), ACM, 2002, pp. 1–10. doi:10.1145/570681.570682.

[32] D. Raffo, C. Adjih, T. Clausen, P. Muhlethaler, An advanced signature system for OLSR, in: Proceedings of the 2nd ACM Workshop on Security of Ad Hoc and Sensor Networks (SASN 2004), ACM, 2004, pp. 10–16. doi:10.1145/1029102.1029105.

[33] R. van Glabbeek, P. Höfner, M. Portmann, W. L. Tan, Modelling and verifying the AODV routing protocol, Distributed Computing 29 (2016) 279–315. doi:10.1007/s00446-015-0262-7.

[34] M. Al-Zyoud, Y. B. Adhyaru, C. Padmini, G. Kaur, P. K. Mishra, B. Goyal, S. K. Samal, J. A. Mayan, Intrusion-resilient intelligent routing in VANETs using dynamic optimization, SN Computer Science 7 (2026) 166.
URL https://link.springer.com/article/10.1007/s42979-026-04768-1

[35] J. Ginesin, M. von Hippel, E. Defloor, C. Nita-Rotaru, M. Tüxen, A formal analysis of SCTP: Attack synthesis and patch verification, in: 33rd USENIX Security Symposium (USENIX Security 24), USENIX Association, 2024, pp. 3099–3116.
URL https://www.usenix.org/conference/usenixsecurity24/presentation/ginesin

[36] X. Song, L. Pei, J. Wu, ProtocolGuard: Detecting protocol non-compliance bugs via LLM-guided static analysis and dynamic verification, in: Network and Distributed System Security Symposium (NDSS 2026), Internet Society, 2026.

[37] Z. Shen, I. Karim, E. Bertino, WCDCAnalyzer: Scalable security analysis of Wi-Fi certified device connectivity protocols, in: Network and

Distributed System Security Symposium (NDSS 2026), Internet Society, 2026.

[38] M. S. Turan, K. A. McKay, D. Chang, J. Kang, J. Kelsey, Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions, Special Publication 800-232, National Institute of Standards and Technology (2025). doi:10.6028/NIST.SP.800-232.
URL https://csrc.nist.gov/pubs/sp/800/232/final

[39] S. Ryczek, M. Sobieraj, Secure LoRa-based transmission system: An IoT solution for smart homes and industries, Electronics 14 (24) (2025) 4977. doi:10.3390/electronics14244977.